

OmniBus Management System

Database Systems Lab Project Report (Spring 2026)

1. Title Page

Project Title: OmniBus Management System

Course: Database Systems Lab (CL2005)

Institution: FAST-NUCES CFD Campus

Semester: Spring 2026

Group Members:

- Muhammad Abdullah 24P-0563
- Salman Shoaib 24P-0583

2. Table of Contents

1. Title Page
 2. Table of Contents
 3. Introduction
 4. Requirements
 5. Database Design (ERD/EERD Explanation)
 6. Schema Description
 7. Query Documentation
 8. PL/SQL Documentation
 9. GUI Documentation
 10. Dashboard Explanation
 11. Challenges & Lessons Learned
 12. Individual Contribution Table
-

3. Introduction

3.1 Domain Description

The OmniBus Management System is a database-driven application designed to manage bus transportation operations including scheduling, ticket booking, passenger management, payment processing, and staff management. The system provides a centralized platform for efficient and organized handling of transportation services.

3.2 Problem Statement

Traditional bus booking systems rely heavily on manual processes, leading to inefficiencies such as booking errors, lack of real-time seat tracking, delayed payments, and poor data management. This project aims to eliminate these issues by implementing a structured database-driven solution.

3.3 Objectives

- Automate bus ticket booking process
 - Manage buses, routes, trips, and passengers
 - Enable secure authentication and role-based access
 - Provide real-time seat availability updates
 - Handle payment and ticket generation
 - Generate analytical reports for decision making
-

4. Requirements

4.1 Functional Requirements

1. System shall allow user registration and login
2. System shall support role-based access control
3. System shall manage bus and route information
4. System shall manage trip scheduling
5. System shall allow seat booking by passengers
6. System shall process payments
7. System shall generate tickets with unique IDs
8. System shall update seat availability dynamically
9. System shall allow booking cancellation
10. System shall generate booking history

4.2 Non-Functional Requirements

- Performance: System shall respond within 2 seconds for most operations
 - Security: User credentials must be securely stored
 - Scalability: System should handle increasing number of users and bookings
-

5. Database Design (ERD / EERD Explanation)

5.1 ER Diagram Overview

The ER diagram consists of entities such as Passenger, Booking, Bus, Trip, Payment, and Ticket. Relationships between entities define how data is interconnected.

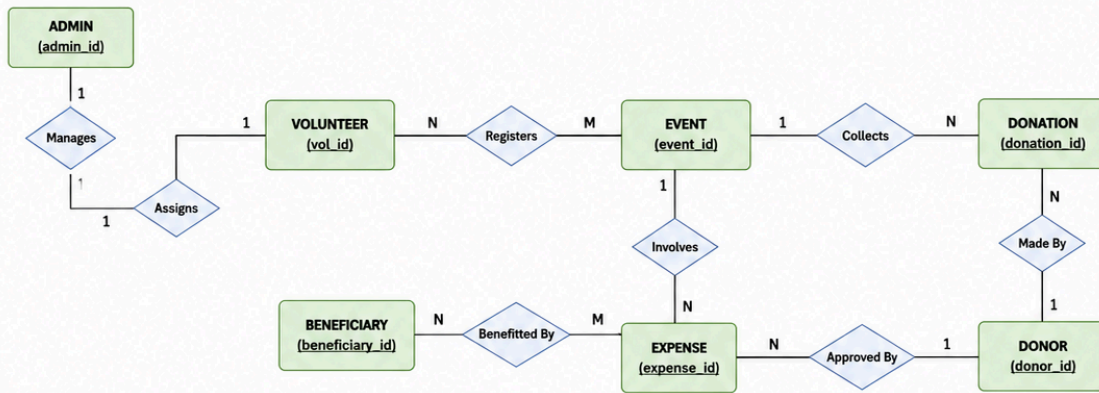
5.2 EER Diagram Features

- Specialization: BusCrew is specialized into Driver, Hostess, SecurityStaff, and Attendant
- Aggregation: Bus and Seat relationship
- Weak Entity: Seat depends on Bus

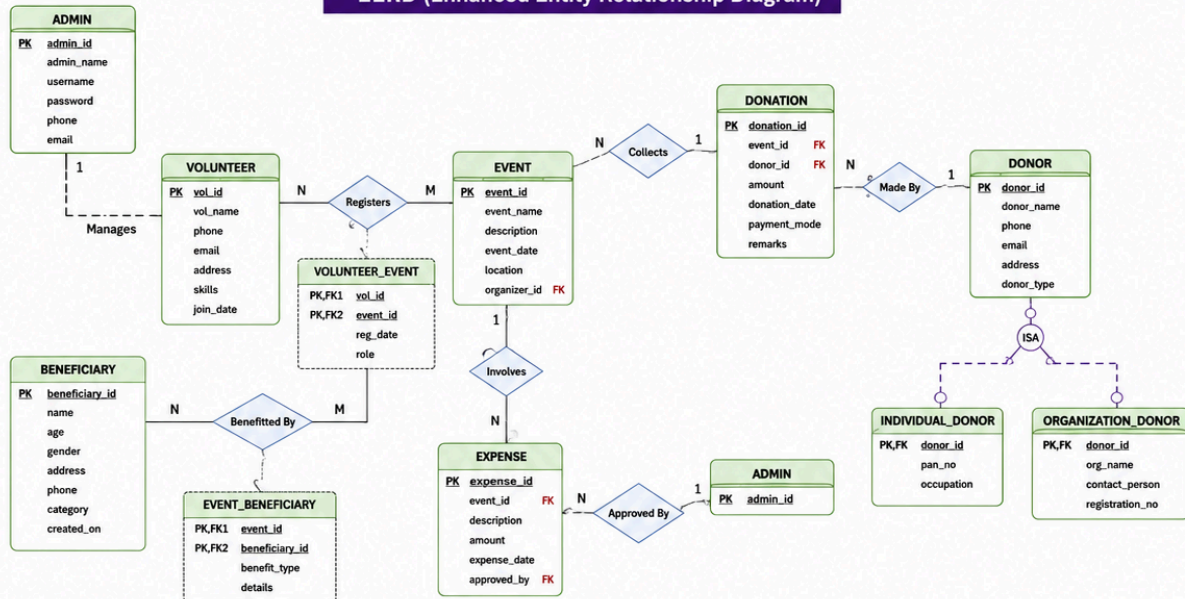
5.3 Relationships

- Passenger → Booking (1:M)
- Booking → Payment (1:1)
- Bus → Seat (1:M)
- Bus → Trip (1:M)
- Route → Trip (1:M)

ERD (Entity Relationship Diagram)



EERD (Enhanced Entity Relationship Diagram)



Notes:

- Many-to-many relationships are resolved using associative entities.
- Specialization is used for DONOR entity into Individual and Organization donors.

Legend

- PK = Primary Key
- FK = Foreign Key
- PK,FK = Primary Key and Foreign Key
- Entity
- Relationship
- Associative Entity
- Inheritance (ISA)

Cardinality Notation

- 1 = One
- N = Many
- M = Many (in many-to-many relationship)

6. Schema Description

6.1 Tables Overview

The system consists of multiple relational tables including:

- Admin
- Passenger
- Booking
- Payment
- Ticket
- Bus
- Route
- Trip
- BusCrew hierarchy

6.2 Normalization

All tables are normalized up to Third Normal Form (3NF) to eliminate redundancy and ensure data integrity.

6.3 Constraints

- Primary Keys defined for all tables
 - Foreign Keys maintain referential integrity
 - CHECK constraints validate data (e.g., phone number format)
 - UNIQUE constraints ensure uniqueness (email, CNIC)
-

7. Query Documentation

```
const express = require('express');

const mysql = require('mysql2');

const cors = require('cors');

const app = express();

app.use(cors());

app.use(express.json());
```

```
const conn = mysql.createConnection({
  host: '127.0.0.1',
  port: 3307,
  user: 'root',
  password: 'abduallahdatabaseproject123@',
  database: 'OmniBus_Management'
});

conn.connect(err => {
  if (err) throw err;
  console.log('MySQL connected!');
});

// Helper: reset AUTO_INCREMENT to MAX(id)+1 after manual ID inserts
function resetAutoIncrement(table, idColumn) {
  return new Promise((resolve) => {
    conn.query(`SELECT MAX(${idColumn}) as maxId FROM ${table}`, (err, rows) => {
      if (err || !rows[0].maxId) return resolve();
      const nextId = parseInt(rows[0].maxId) + 1;
      conn.query(`ALTER TABLE ${table} AUTO_INCREMENT = ${nextId}`, () =>
        resolve());
    });
  });
}
```

```
// — AUTHENTICATION
```

```
app.post('/api/auth/signup', (req, res) => {  
  const { Username, Password } = req.body;  
  
  if (!Username || !Password) return res.status(400).json({ error: 'Username and  
password required' });  
  
  conn.query('INSERT INTO Admin (Username, Password) VALUES (?, ?)', [Username,  
Password], (err, r) => {  
    if (err) {  
      if (err.code === 'ER_DUP_ENTRY') return res.status(409).json({ error: 'Username already exists' });  
      return res.status(500).json({ error: err.message });  
    }  
    res.json({ message: 'Signup successful' });  
  });  
});
```

```
app.post('/api/auth/login', (req, res) => {  
  const { Username, Password } = req.body;  
  
  if (!Username || !Password) return res.status(400).json({ error: 'Username and  
password required' });  
  
  conn.query('SELECT * FROM Admin WHERE Username = ? AND Password = ?',  
[Username, Password], (err, r) => {  
    if (err) return res.status(500).json({ error: err.message });  
    if (r.length === 0) return res.status(401).json({ error: 'Invalid  
credentials' });  
    res.json({ message: 'Login successful', username: Username });  
  });  
});
```



```

    });

});

// ——— PASSENGERS

app.get('/api/passengers', (req, res) => {

    conn.query('SELECT * FROM Passenger', (err, r) => err ? res.status(500).json({
error: err.message }) : res.json(r));

});

app.get('/api/passengers/:id', (req, res) => {

    conn.query('SELECT * FROM Passenger WHERE PassengerID = ?', [req.params.id],
(err, r) => err ? res.status(500).json({ error: err.message }) : res.json(r[0]));

});

app.post('/api/passengers', async (req, res) => {

    const { PassengerID, FirstName, LastName, Email, Phone, CNIC, DateOfBirth,
Password } = req.body;

    conn.query('INSERT INTO Passenger VALUES (?, ?, ?, ?, ?, ?, ?, ?)', [PassengerID ||
null, FirstName, LastName, Email, Phone, CNIC, DateOfBirth, Password],

    async (err, r) => {

        if (err) return res.status(500).json({ error: err.message });

        await resetAutoIncrement('Passenger', 'PassengerID');

        res.json({ message: 'Passenger created', id: r.insertId });

    });

});

app.put('/api/passengers/:id', (req, res) => {

    const { FirstName, LastName, Email, Phone, CNIC, DateOfBirth } = req.body;

```

```
    conn.query('UPDATE Passenger SET FirstName=?, LastName=?, Email=?, Phone=?,
CNIC=?, DateOfBirth=? WHERE PassengerID=?',

    [FirstName, LastName, Email, Phone, CNIC, DateOfBirth, req.params.id],

    (err) => err ? res.status(500).json({ error: err.message }) : res.json({
message: 'Updated' }));

});

app.delete('/api/passengers/:id', (req, res) => {

    conn.query('DELETE FROM Passenger WHERE PassengerID=?', [req.params.id], (err)
=> err ? res.status(500).json({ error: err.message }) : res.json({ message:
'Deleted' }));

});

// ——— BUS TYPES
```

```
app.post('/api/bustypes', (req, res) => {

    const { BusID, CategoryName } = req.body;

    conn.query('INSERT IGNORE INTO BusType VALUES (?,?)', [BusID, CategoryName],

    (err) => err ? res.status(500).json({ error: err.message }) : res.json({
message: 'BusType added' }));

});

app.delete('/api/bustypes/:busId', (req, res) => {

    conn.query('DELETE FROM BusType WHERE BusID=?', [req.params.busId],

    (err) => err ? res.status(500).json({ error: err.message }) : res.json({
message: 'BusTypes cleared' }));

});
```

```
// — BUSES
```

```
app.get('/api/buses', (req, res) => {

  conn.query(`SELECT b.*, bc.Name as DriverName, hc.Name as HostessName,
GROUP_CONCAT(bt.CategoryName ORDER BY bt.CategoryName) as Types

      FROM Bus b

      LEFT JOIN BusCrew bc ON b.OperatorID = bc.OperatorID

      LEFT JOIN BusCrew hc ON b.HostessID = hc.OperatorID

      LEFT JOIN BusType bt ON b.BusID = bt.BusID

      GROUP BY b.BusID`, (err, r) => err ? res.status(500).json({ error:
err.message }) : res.json(r));

});

app.post('/api/buses', async (req, res) => {

  const { BusID, BusNumber, TotalSeats, OperatorID, HostessID, Types } =
req.body;

  conn.query('INSERT INTO Bus (BusID, BusNumber, TotalSeats, OperatorID,
HostessID) VALUES (?, ?, ?, ?, ?)', [BusID || null, BusNumber, TotalSeats, OperatorID
|| null, HostessID || null],

    async (err, result) => {

      if (err) return res.status(500).json({ error: err.message });

      // FIX 3: Auto-generate Seat rows for the new bus

      const actualBusId = BusID || result.insertId;

      const totalSeats = parseInt(TotalSeats) || 0;

      if (totalSeats > 0) {

        const seatValues = [];

        const windowCount = Math.round(totalSeats * 0.2);
```

```

const frontCount = Math.round(totalSeats * 0.2);

const backCount = Math.round(totalSeats * 0.2);

for (let i = 1; i <= totalSeats; i++) {

  let seatType = 'Standard';

  if (i <= frontCount) {

    seatType = 'Front Seat';

  } else if (i <= frontCount + windowCount) {

    seatType = 'Window Seat';

  } else if (i <= frontCount + windowCount + backCount) {

    seatType = 'Back Seat';

  }

  seatValues.push([null, actualBusId, `Seat-${i}`, seatType,
'Available']);

}

conn.query('INSERT INTO Seat (SeatID, BusID, SeatNumber, SeatType,
SeatStatus) VALUES ?',

[seatValues], (seatErr) => {

  if (seatErr) console.error('Seat generation error:',
seatErr.message);

});

}

await resetAutoIncrement('Bus', 'BusID');

res.json({ message: 'Bus created', id: actualBusId });

});

```

```

});

app.put('/api/buses/:id', (req, res) => {

  const { BusNumber, TotalSeats, OperatorID, HostessID } = req.body;

  conn.query('UPDATE Bus SET BusNumber=?, TotalSeats=?, OperatorID=?, HostessID=?
WHERE BusID=?',

    [BusNumber, TotalSeats, OperatorID || null, HostessID || null,
req.params.id],

    (err) => {

      if (err) return res.status(500).json({ error: err.message });

      res.json({ message: 'Updated' });

    });

});

app.delete('/api/buses/:id', (req, res) => {

  conn.query('DELETE FROM Bus WHERE BusID=?', [req.params.id], (err) => err ?
res.status(500).json({ error: err.message }) : res.json({ message: 'Deleted' }));

});

// — ROUTES

app.get('/api/routes', (req, res) => {

  conn.query('SELECT * FROM Route', (err, r) => err ? res.status(500).json({
error: err.message }) : res.json(r));

});

app.post('/api/routes', async (req, res) => {

  const { RouteID, SourceCity, DestinationCity, Distance, EstimatedDuration } =
req.body;

```

```

    conn.query('INSERT INTO Route VALUES (?, ?, ?, ?, ?)', [RouteID || null,
SourceCity, DestinationCity, Distance, EstimatedDuration],

    async (err) => {

        if (err) return res.status(500).json({ error: err.message });

        await resetAutoIncrement('Route', 'RouteID');

        res.json({ message: 'Route created' });

    });

});

app.put('/api/routes/:id', (req, res) => {

    const { SourceCity, DestinationCity, Distance, EstimatedDuration } = req.body;

    conn.query('UPDATE Route SET SourceCity=?, DestinationCity=?, Distance=?,
EstimatedDuration=? WHERE RouteID=?',

        [SourceCity, DestinationCity, Distance, EstimatedDuration, req.params.id],

        (err) => err ? res.status(500).json({ error: err.message }) : res.json({
message: 'Updated' }));

});

app.delete('/api/routes/:id', (req, res) => {

    conn.query('DELETE FROM Route WHERE RouteID=?', [req.params.id], (err) => err ?
res.status(500).json({ error: err.message }) : res.json({ message: 'Deleted' }));

});

// ——— TRIPS

app.get('/api/trips', (req, res) => {

    conn.query(`SELECT t.*, b.BusNumber, r.SourceCity, r.DestinationCity

        FROM Trip t

```

```

        LEFT JOIN Bus b ON t.BusID = b.BusID

        LEFT JOIN Route r ON t.RouteID = r.RouteID`, (err, r) => err ?
res.status(500).json({ error: err.message }) : res.json(r));
});

app.post('/api/trips', async (req, res) => {

    const { TripID, BusID, RouteID, DepartureDate, DepartureTime, ArrivalTime, Fare
} = req.body;

    conn.query('INSERT INTO Trip VALUES (?, ?, ?, ?, ?, ?, ?)', [TripID || null, BusID,
RouteID, DepartureDate, DepartureTime, ArrivalTime, Fare],

        async (err) => {

            if (err) return res.status(500).json({ error: err.message });

            await resetAutoIncrement('Trip', 'TripID');

            res.json({ message: 'Trip created' });

        });

});

app.put('/api/trips/:id', (req, res) => {

    const { BusID, RouteID, DepartureDate, DepartureTime, ArrivalTime, Fare } =
req.body;

    conn.query('UPDATE Trip SET BusID=?, RouteID=?, DepartureDate=?,
DepartureTime=?, ArrivalTime=?, Fare=? WHERE TripID=?',

        [BusID, RouteID, DepartureDate, DepartureTime, ArrivalTime, Fare,
req.params.id],

        (err) => err ? res.status(500).json({ error: err.message }) : res.json({
message: 'Updated' }));

});

app.delete('/api/trips/:id', (req, res) => {

```

```
    conn.query('DELETE FROM Trip WHERE TripID=?', [req.params.id], (err) => err ?
res.status(500).json({ error: err.message }) : res.json({ message: 'Deleted' }));
});
```

```
// — PL/SQL: STORED PROCEDURE ENDPOINTS
```

```
app.get('/api/trips/:id/revenue', (req, res) => {
    conn.query('CALL GetTripRevenue(?, @total)', [req.params.id], (err) => {
        if (err) return res.status(500).json({ error: err.message });
        conn.query('SELECT @total AS TotalRevenue', (err2, results) => {
            if (err2) return res.status(500).json({ error: err2.message });
            res.json({ TripID: req.params.id, Revenue: results[0].TotalRevenue });
        });
    });
});
```

```
// — BOOKINGS
```

```
app.get('/api/bookings', (req, res) => {
    conn.query(`SELECT bk.*, CONCAT(p.FirstName, ' ', p.LastName) as PassengerName,
        r.SourceCity, r.DestinationCity, t.DepartureDate
    FROM Booking bk
    LEFT JOIN Passenger p ON bk.PassengerID = p.PassengerID
    LEFT JOIN Trip t ON bk.TripID = t.TripID
```



```

        LEFT JOIN Route r ON t.RouteID = r.RouteID`, (err, r) => err ?
res.status(500).json({ error: err.message }) : res.json(r));
});

app.post('/api/bookings', async (req, res) => {

    const { BookingID, PassengerID, TripID, TotalAmount, BookingStatus } =
req.body;

    conn.query('INSERT INTO Booking (BookingID, PassengerID, TripID, TotalAmount,
BookingStatus) VALUES (?, ?, ?, ?, ?)',

        [BookingID || null, PassengerID, TripID, TotalAmount, BookingStatus ||
'Pending'],

        async (err) => {

            if (err) return res.status(500).json({ error: err.message });

            await resetAutoIncrement('Booking', 'BookingID');

            res.json({ message: 'Booking created' });

        });

});

app.put('/api/bookings/:id', (req, res) => {

    const { PassengerID, TripID, TotalAmount, BookingStatus } = req.body;

    conn.query('UPDATE Booking SET PassengerID=?, TripID=?, TotalAmount=?,
BookingStatus=? WHERE BookingID=?',

        [PassengerID, TripID, TotalAmount, BookingStatus, req.params.id],

        (err) => err ? res.status(500).json({ error: err.message }) : res.json({
message: 'Updated' }));

});

app.delete('/api/bookings/:id', (req, res) => {

```

```

    conn.query('DELETE FROM Booking WHERE BookingID=?', [req.params.id], (err) =>
err ? res.status(500).json({ error: err.message }) : res.json({ message:
'Deleted' }));

});

// ——— PAYMENTS

app.get('/api/payments', (req, res) => {

    conn.query(`SELECT py.*, bk.TotalAmount, bk.BookingStatus,

        CONCAT(p.FirstName, ' ',p.LastName) as PassengerName

        FROM Payment py

        LEFT JOIN Booking bk ON py.BookingID = bk.BookingID

        LEFT JOIN Passenger p ON bk.PassengerID = p.PassengerID`, (err, r)
=> err ? res.status(500).json({ error: err.message }) : res.json(r));

});

app.post('/api/payments', async (req, res) => {

    const { PaymentID, BookingID, PaymentMethod, PaymentAmount, PaymentStatus } =
req.body;

    conn.query('INSERT INTO Payment (PaymentID, BookingID, PaymentMethod,
PaymentAmount, PaymentStatus) VALUES (?, ?, ?, ?, ?)',

        [PaymentID || null, BookingID, PaymentMethod, PaymentAmount, PaymentStatus ||
'Pending'],

        async (err, result) => {

            if (err) return res.status(500).json({ error: err.message });

            // Auto-update booking status based on payment status

```

```

    let bookingStatus = 'Pending';

    if (PaymentStatus === 'Completed') bookingStatus = 'Confirmed';

    else if (PaymentStatus === 'Failed') bookingStatus = 'Cancelled';

    conn.query('UPDATE Booking SET BookingStatus = ? WHERE BookingID = ?',
[bookingStatus, BookingID], (updErr) => {

        if (updErr) console.error('Booking status sync error:', updErr.message);

    });

    await resetAutoIncrement('Payment', 'PaymentID');

    res.json({ message: 'Payment recorded' });

});

});

app.put('/api/payments/:id', (req, res) => {

    const { PaymentMethod, PaymentAmount, PaymentStatus } = req.body;

    conn.query('UPDATE Payment SET PaymentMethod=?, PaymentAmount=?,
PaymentStatus=? WHERE PaymentID=?',

    [PaymentMethod, PaymentAmount, PaymentStatus, req.params.id],

    (err) => {

        if (err) return res.status(500).json({ error: err.message });

        // Sync with Booking status

        conn.query('SELECT BookingID FROM Payment WHERE PaymentID = ?',
[req.params.id], (errB, rows) => {

            if (!errB && rows.length > 0) {

```

```

        const bId = rows[0].BookingID;

        let bStatus = 'Pending';

        if (PaymentStatus === 'Completed') bStatus = 'Confirmed';

        else if (PaymentStatus === 'Failed') bStatus = 'Cancelled';

        conn.query('UPDATE Booking SET BookingStatus = ? WHERE BookingID = ?',
[bStatus, bId]);

    }

    });

    res.json({ message: 'Updated' });

    });

});

app.delete('/api/payments/:id', (req, res) => {

    conn.query('DELETE FROM Payment WHERE PaymentID=?', [req.params.id], (err) =>
err ? res.status(500).json({ error: err.message }) : res.json({ message:
'Deleted' }));

});

// ——— STAFF (BusCrew)

app.get('/api/staff', (req, res) => {

    conn.query(`SELECT bc.*,

        d.LicenseNumber as DriverLicense, d.ExperienceYears,

        ss.LicenseNumber as SecurityLicense,

        a.ServiceLevel,

```

```

h.LanguagesKnown,

CASE

    WHEN d.OperatorID IS NOT NULL THEN 'Driver'

    WHEN ss.OperatorID IS NOT NULL THEN 'Security'

    WHEN a.OperatorID IS NOT NULL THEN 'Attendant'

    WHEN h.OperatorID IS NOT NULL THEN 'Hostess'

    ELSE 'Staff'

END as Role

FROM BusCrew bc

LEFT JOIN Driver d ON bc.OperatorID = d.OperatorID

LEFT JOIN SecurityStaff ss ON bc.OperatorID = ss.OperatorID

LEFT JOIN Attendant a ON bc.OperatorID = a.OperatorID

LEFT JOIN Hostess h ON bc.OperatorID = h.OperatorID`, (err, r) => err ?
res.status(500).json({ error: err.message }) : res.json(r));
});

// server.js mein Staff post route ko update karein

app.post('/api/staff', async (req, res) => {

    const { OperatorID, Name, PhoneNumber, Address, Role, LicenseNumber,
    SLicenseNumber, ExperienceYears, ServiceLevel, Languages } = req.body;

    conn.query('INSERT INTO BusCrew (OperatorID, Name, PhoneNumber, Address) VALUES
    (?, ?, ?, ?)',

    [OperatorID || null, Name, PhoneNumber, Address], async (err, result) => {

        if (err) return res.status(500).json({ error: err.message });

        await resetAutoIncrement('BusCrew', 'OperatorID');

```

```

const actualOpId = OperatorID || result.insertId;

if (Role === 'Driver') {

    // Driver.LicenseNumber gets the driver license

    conn.query('INSERT INTO Driver (OperatorID, LicenseNumber,
ExperienceYears) VALUES (?, ?, ?)',

        [actualOpId, LicenseNumber, ExperienceYears],

        (err2) => { if (err2) return res.status(500).json({ error: err2.message
}); res.json({ message: 'Staff added successfully' }); });

} else if (Role === 'Hostess') {

    conn.query('INSERT INTO Hostess (OperatorID, LanguagesKnown) VALUES
(?, ?)', [actualOpId, Languages || ''], (err2) => {

        if (err2) return res.status(500).json({ error: err2.message });

        res.json({ message: 'Staff added successfully' });

    });

} else if (Role === 'Security') {

    // FIX 2: SecurityStaff.LicenseNumber gets SLicenseNumber (the
security-specific field)

    conn.query('INSERT INTO SecurityStaff (OperatorID, LicenseNumber) VALUES
(?, ?)',

        [actualOpId, SLicenseNumber],

        (err2) => { if (err2) return res.status(500).json({ error: err2.message
}); res.json({ message: 'Staff added successfully' }); });

} else if (Role === 'Attendant') {

```

```

        conn.query('INSERT INTO Attendant (OperatorID, ServiceLevel) VALUES
(?,?)',

        [actualOpId, ServiceLevel],

        (err2) => { if (err2) return res.status(500).json({ error: err2.message
}); res.json({ message: 'Staff added successfully' }); });

    } else {

        res.json({ message: 'Staff added successfully' });

    }

});

});

app.put('/api/staff/:id', (req, res) => {

    const { Name, PhoneNumber, Address, Role, LicenseNumber, SLicenseNumber,
ExperienceYears, ServiceLevel, Languages } = req.body;

    const operatorID = req.params.id;

    conn.query('UPDATE BusCrew SET Name=?, PhoneNumber=?, Address=? WHERE
OperatorID=?',

    [Name, PhoneNumber, Address, operatorID],

    (err) => {

        if (err) return res.status(500).json({ error: err.message });

        if (Role === 'Driver') {

            conn.query('UPDATE Driver SET LicenseNumber=?, ExperienceYears=? WHERE
OperatorID=?', [LicenseNumber, ExperienceYears, operatorID], (err2) => {

                if (err2) return res.status(500).json({ error: err2.message });

```

```
    res.json({ message: 'Updated' });

  });

  } else if (Role === 'Hostess') {

    conn.query('UPDATE Hostess SET LanguagesKnown=? WHERE OperatorID=?',
[Languages || '', operatorID], (err2) => {

      if (err2) return res.status(500).json({ error: err2.message });

      res.json({ message: 'Updated' });

    });

  } else if (Role === 'Security') {

    conn.query('UPDATE SecurityStaff SET LicenseNumber=? WHERE OperatorID=?',
[SLicenseNumber, operatorID], (err2) => {

      if (err2) return res.status(500).json({ error: err2.message });

      res.json({ message: 'Updated' });

    });

  } else if (Role === 'Attendant') {

    conn.query('UPDATE Attendant SET ServiceLevel=? WHERE OperatorID=?',
[ServiceLevel, operatorID], (err2) => {

      if (err2) return res.status(500).json({ error: err2.message });

      res.json({ message: 'Updated' });

    });

  } else {

    res.json({ message: 'Updated' });

  }

});

});
```



```
app.delete('/api/staff/:id', (req, res) => {  
    conn.query('DELETE FROM BusCrew WHERE OperatorID=?', [req.params.id], (err) =>  
err ? res.status(500).json({ error: err.message }) : res.json({ message:  
'Deleted' }));  
});
```

```
// — SEATS
```

```
app.get('/api/seats', (req, res) => {  
    const { bookingId } = req.query;  
    if (bookingId) {  
        const q = `  
            SELECT s.*, b.BusNumber  
            FROM Seat s  
            JOIN Bus b ON s.BusID = b.BusID  
            JOIN Trip t ON b.BusID = t.BusID  
            JOIN Booking bk ON t.TripID = bk.TripID  
            WHERE bk.BookingID = ? AND s.SeatStatus = 'Available'`;   
        conn.query(q, [bookingId], (err, r) => err ? res.status(500).json({ error:  
err.message }) : res.json(r));  
    } else {  
        conn.query(`SELECT s.*, b.BusNumber FROM Seat s LEFT JOIN Bus b ON s.BusID =  
b.BusID WHERE s.SeatStatus = 'Available'`, (err, r) => err ?  
res.status(500).json({ error: err.message }) : res.json(r));  
    }  
});
```

```
// ——— TICKETS
```

```
app.get('/api/tickets', (req, res) => {

  conn.query(`SELECT tk.*, s.SeatNumber, b.BusNumber,

    CONCAT(p.FirstName, ' ', p.LastName) as PassengerName,

    r.SourceCity, r.DestinationCity

  FROM Ticket tk

  LEFT JOIN Booking bk ON tk.BookingID = bk.BookingID

  LEFT JOIN Seat s ON tk.SeatID = s.SeatID

  LEFT JOIN Passenger p ON bk.PassengerID = p.PassengerID

  LEFT JOIN Trip t ON bk.TripID = t.TripID

  LEFT JOIN Bus b ON t.BusID = b.BusID

  LEFT JOIN Route r ON t.RouteID = r.RouteID`, (err, r) => err ?
res.status(500).json({ error: err.message }) : res.json(r));

});

app.post('/api/tickets', async (req, res) => {

  const { TicketID, BookingID, SeatID, TicketNumber, QRCode } = req.body;

  conn.query('SELECT TicketID FROM Ticket WHERE TicketNumber = ?',
[TicketNumber], (checkErr, existing) => {

    if (checkErr) return res.status(500).json({ error: checkErr.message });

    if (existing.length > 0) {

      return res.status(409).json({ error: `Ticket number "${TicketNumber}"
already exists. Please use a unique ticket number.` });

    }

  })

});
```

```

    conn.query('INSERT INTO Ticket (TicketID, BookingID, SeatID, TicketNumber,
QRCode) VALUES (?, ?, ?, ?, ?)',

    [TicketID || null, BookingID, SeatID, TicketNumber, QRCode || null],

    async (err, result) => {

        if (err) return res.status(500).json({ error: err.message });

        await resetAutoIncrement('Ticket', 'TicketID');

        res.json({ message: 'Ticket issued', TicketID: result.insertId ||
TicketID });

    });

});

app.delete('/api/tickets/:id', (req, res) => {

    conn.query('DELETE FROM Ticket WHERE TicketID=?', [req.params.id], (err) => err
? res.status(500).json({ error: err.message }) : res.json({ message: 'Deleted'
})));

});

// ——— DASHBOARD STATS

app.get('/api/stats', (req, res) => {

    const queries = {

        passengers: 'SELECT COUNT(*) as count FROM Passenger',

        buses: 'SELECT COUNT(*) as count FROM Bus',

        trips: 'SELECT COUNT(*) as count FROM Trip',

        bookings: 'SELECT COUNT(*) as count FROM Booking',

        revenue: 'SELECT COALESCE(SUM(PaymentAmount),0) as total FROM Payment WHERE
PaymentStatus="Completed"',

```

```

    pendingBookings: 'SELECT COUNT(*) as count FROM Booking WHERE
BookingStatus="Pending"

};

const results = {};

let done = 0;

const keys = Object.keys(queries);

keys.forEach(key => {

    conn.query(queries[key], (err, r) => {

        if (!err) results[key] = r[0].count ?? r[0].total;

        if (++done === keys.length) res.json(results);

    });

});

});

app.listen(3000, () => console.log('Server running on http://localhost:3000'));

```

8. PL/SQL Documentation

```

DROP DATABASE IF EXISTS OmniBus_Management;

CREATE DATABASE OmniBus_Management;

USE OmniBus_Management;

-- Admin Users for Portal

CREATE TABLE Admin (

```

```

AdminID INT AUTO_INCREMENT PRIMARY KEY,

Username VARCHAR(50) UNIQUE NOT NULL,

Password VARCHAR(255) NOT NULL,

CreatedAt DATETIME DEFAULT CURRENT_TIMESTAMP

);

-- Staff Hierarchy

CREATE TABLE BusCrew(

    OperatorID INT AUTO_INCREMENT PRIMARY KEY,

    Name VARCHAR(100),

    PhoneNumber VARCHAR(11) CHECK (PhoneNumber REGEXP '^[0-9]{11}$'),

    Address VARCHAR(100)

);

CREATE TABLE Driver(

    OperatorID INT,

    LicenseNumber VARCHAR(50),

    ExperienceYears INT,

    PRIMARY KEY (OperatorID),

    CONSTRAINT FK_Driver_BusCrew

    FOREIGN KEY (OperatorID) REFERENCES BusCrew(OperatorID)

    ON DELETE CASCADE

    ON UPDATE CASCADE

);

```

```
CREATE TABLE Hostess(  
    OperatorID INT,  
    LanguagesKnown VARCHAR(100),  
    PRIMARY KEY (OperatorID),  
    CONSTRAINT FK_Hostess_BusCrew  
    FOREIGN KEY (OperatorID) REFERENCES BusCrew(OperatorID)  
    ON DELETE CASCADE  
    ON UPDATE CASCADE  
);
```

```
CREATE TABLE SecurityStaff(  
    OperatorID INT,  
    PRIMARY KEY (OperatorID),  
    LicenseNumber VARCHAR(50),  
    CONSTRAINT FK_SecurityStaff_BusCrew  
    FOREIGN KEY (OperatorID) REFERENCES BusCrew(OperatorID)  
    ON DELETE CASCADE  
    ON UPDATE CASCADE  
);
```

```
CREATE TABLE Attendant(  
    OperatorID INT,  
    PRIMARY KEY (OperatorID),  
    ServiceLevel ENUM('Trainee', 'Junior', 'Senior', 'Expert') DEFAULT 'Junior',  
    CONSTRAINT FK_Attendant_BusCrew
```

```

FOREIGN KEY (OperatorID) REFERENCES BusCrew(OperatorID)

ON DELETE CASCADE

ON UPDATE CASCADE

);

-- Vehicles and Assets

CREATE TABLE Bus (

    BusID INT AUTO_INCREMENT PRIMARY KEY,

    BusNumber VARCHAR(20) UNIQUE,

    TotalSeats INT,

    OperatorID INT,

    CONSTRAINT FK_Bus_BusCrew

    FOREIGN KEY (OperatorID) REFERENCES BusCrew(OperatorID)

    ON DELETE SET NULL

    ON UPDATE CASCADE

);

CREATE TABLE BusType (

    BusID INT,

    CategoryName ENUM('Luxury', 'AC', 'Sleeper'),

    PRIMARY KEY (BusID, CategoryName),

    CONSTRAINT FK_BusType_Bus

    FOREIGN KEY (BusID) REFERENCES Bus(BusID)

    ON DELETE CASCADE

    ON UPDATE CASCADE

```

```

);

CREATE TABLE Seat (
    SeatID INT AUTO_INCREMENT PRIMARY KEY,
    BusID INT,
    SeatNumber VARCHAR(10),
    SeatType VARCHAR(20),
    SeatStatus VARCHAR(20) DEFAULT 'Available',
    CONSTRAINT FK_Seat_Bus
    FOREIGN KEY (BusID) REFERENCES Bus(BusID)
    ON DELETE CASCADE
    ON UPDATE CASCADE
);

-- Operations and Scheduling

CREATE TABLE Route (
    RouteID INT AUTO_INCREMENT PRIMARY KEY,
    SourceCity VARCHAR(100) NOT NULL,
    DestinationCity VARCHAR(100) NOT NULL,
    Distance DECIMAL(6,2),
    EstimatedDuration TIME
);

CREATE TABLE Trip (
    TripID INT AUTO_INCREMENT PRIMARY KEY,

```



```

    BusID INT,

    RouteID INT,

    DepartureDate DATE,

    DepartureTime TIME,

    ArrivalTime TIME,

    Fare DECIMAL(10,2),

    CONSTRAINT FK_Trip_Bus FOREIGN KEY (BusID) REFERENCES Bus(BusID) ON DELETE
CASCADE,

    CONSTRAINT FK_Trip_Route FOREIGN KEY (RouteID) REFERENCES Route(RouteID) ON
DELETE CASCADE

);

-- Transactional Entities

CREATE TABLE Passenger (

    PassengerID INT AUTO_INCREMENT PRIMARY KEY,

    FirstName VARCHAR(50),

    LastName VARCHAR(50),

    Email VARCHAR(100) UNIQUE,

    Phone VARCHAR(11) CHECK (Phone REGEXP '^[0-9]{11}$'),

    CNIC VARCHAR(13) CHECK (CNIC REGEXP '^[0-9]{13}$') UNIQUE,

    DateOfBirth DATE,

    Password VARCHAR(255)

);

CREATE TABLE Booking (

    BookingID INT AUTO_INCREMENT PRIMARY KEY,

```

```
    PassengerID INT,

    TripID INT,

    BookingDate DATETIME DEFAULT CURRENT_TIMESTAMP,

    TotalAmount DECIMAL(10,2),

    BookingStatus ENUM('Pending', 'Confirmed', 'Cancelled') DEFAULT 'Pending',

    CONSTRAINT FK_Booking_Passenger FOREIGN KEY (PassengerID) REFERENCES
Passenger(PassengerID) ON DELETE CASCADE,

    CONSTRAINT FK_Booking_Trip FOREIGN KEY (TripID) REFERENCES Trip(TripID) ON
DELETE CASCADE

);
```

```
CREATE TABLE Payment (

    PaymentID INT AUTO_INCREMENT PRIMARY KEY,

    BookingID INT UNIQUE,

    PaymentMethod ENUM('Credit Card', 'Debit Card', 'JazzCash', 'EasyPaisa',
'Cash'),

    PaymentAmount DECIMAL(10,2),

    PaymentDate DATETIME DEFAULT CURRENT_TIMESTAMP,

    PaymentStatus ENUM('Pending', 'Completed', 'Failed') DEFAULT 'Pending',

    CONSTRAINT FK_Payment_Booking FOREIGN KEY (BookingID) REFERENCES
Booking(BookingID) ON DELETE CASCADE

);
```

```
CREATE TABLE Ticket (

    TicketID INT AUTO_INCREMENT PRIMARY KEY,

    BookingID INT,

    SeatID INT,
```

```
TicketNumber VARCHAR(50) UNIQUE,  
  
QRCode VARCHAR(255),  
  
IssueDate DATETIME DEFAULT CURRENT_TIMESTAMP,  
  
CONSTRAINT FK_Ticket_Booking FOREIGN KEY (BookingID) REFERENCES  
Booking(BookingID) ON DELETE CASCADE,  
  
CONSTRAINT FK_Ticket_Seat FOREIGN KEY (SeatID) REFERENCES Seat(SeatID) ON  
DELETE CASCADE  
);
```

```
-- ——— PL/SQL: STORED PROCEDURES & TRIGGERS
```

```
DELIMITER //
```

```
-- Procedure to calculate total revenue for a specific trip
```

```
CREATE PROCEDURE GetTripRevenue(IN p_TripID INT, OUT p_TotalRevenue  
DECIMAL(10,2))
```

```
BEGIN
```

```
    SELECT COALESCE(SUM(p.PaymentAmount), 0) INTO p_TotalRevenue
```

```
    FROM Payment p
```

```
    JOIN Booking b ON p.BookingID = b.BookingID
```

```
    WHERE b.TripID = p_TripID AND p.PaymentStatus = 'Completed';
```

```
END //
```

```
-- Trigger to mark seat as booked when a ticket is issued
```

```
CREATE TRIGGER AfterTicketInsert
```

```
AFTER INSERT ON Ticket
```

```
FOR EACH ROW
```

```
BEGIN

    UPDATE Seat

    SET SeatStatus = 'Booked'

    WHERE SeatID = NEW.SeatID;

END //
```

-- Trigger to mark seat as available when a ticket is cancelled/deleted

```
CREATE TRIGGER AfterTicketDelete

AFTER DELETE ON Ticket

FOR EACH ROW

BEGIN

    UPDATE Seat

    SET SeatStatus = 'Available'

    WHERE SeatID = OLD.SeatID;

END //
```

DELIMITER ;

-- Trigger to update Booking status when Payment status changes

```
DELIMITER //
```

```
CREATE TRIGGER AfterPaymentStatusUpdate

AFTER UPDATE ON Payment

FOR EACH ROW

BEGIN

    IF NEW.PaymentStatus <> OLD.PaymentStatus THEN
```

```

        IF NEW.PaymentStatus = 'Completed' THEN

            UPDATE Booking SET BookingStatus = 'Confirmed' WHERE BookingID =
NEW.BookingID;

            ELSEIF NEW.PaymentStatus = 'Pending' THEN

                UPDATE Booking SET BookingStatus = 'Pending' WHERE BookingID =
NEW.BookingID;

                ELSEIF NEW.PaymentStatus = 'Failed' THEN

                    UPDATE Booking SET BookingStatus = 'Cancelled' WHERE BookingID =
NEW.BookingID;

                END IF;

            END IF;

        END //

DELIMITER ;

ALTER TABLE Bus ADD COLUMN HostessID INT NULL,

ADD CONSTRAINT FK_Bus_Hostess FOREIGN KEY (HostessID) REFERENCES
BusCrew(OperatorID) ON DELETE SET NULL ON UPDATE CASCADE;

```

11. Challenges & Lessons Learned

Challenges:

- Designing normalized relational schema
- Managing multiple relationships between entities
- Implementing triggers for real-time updates
- Connecting GUI with database backend

Lessons Learned:

- Practical understanding of relational database design
- SQL and PL/SQL programming skills

- Importance of normalization and constraints
 - Real-world application development workflow
-

END OF REPORT